

A 16-Bit-Native Compression Pipeline for Lossless Medical Images

Kuldeep Singh

A plain-language version of the MIC paper. It keeps the main ideas, the key numbers, and the reasoning, but trades the formal equations and implementation minutiae for explanations a software engineer can follow. Data tables are reproduced from the full IEEE TMI manuscript; figures are omitted.

Abstract

Medical images have to be stored and transmitted without losing a single pixel: a radiologist reading a scan must see exactly what the scanner produced. These images are also “deep” — each pixel is 10 to 16 bits, not the 8 bits of an ordinary photo. That deep pixel format is where most compression tools stumble, because almost all of them were built for 8-bit data.

This paper describes **MIC** (Medical Image Codec), a lossless codec that works *directly* on 16-bit data instead of chopping each pixel into two bytes. MIC chains together three simple stages: a spatial predictor, a 16-bit run-length encoder, and a large-alphabet entropy coder called FSE (Finite State Entropy, a fast table-driven form of arithmetic-style coding). It also includes a “multi-state” decoder trick that keeps the CPU busier and speeds up decompression.

On 21 real DICOM images across nine modalities, MIC:

- compresses **5–22% better** (14% on average) than a strong general-purpose baseline (Delta + Zstandard);
- reaches an average compression ratio of **3.46**, basically tied with High-Throughput JPEG 2000 (HTJ2K, 3.45) and about 91% of JPEG-LS (3.82);
- is the **fastest decoder** of the four codecs tested on 20 of 21 images on ARM64 and on all 21 on AMD64;
- ships with a tiny **20 KB pure-JavaScript decoder** so the format can be decoded right in a web browser.

The main caveat: this is a 21-image study. The results are encouraging but need confirmation on larger, multi-hospital datasets.

1 Why Medical Images Need a Different Codec

A modern scanner produces a lot of data. One digital breast tomosynthesis view alone can be 60+ image slices and over 600 MB; a full screening study tops 2 GB uncompressed. Hospitals store and

move enormous volumes of this data every day, so good lossless compression directly affects storage cost, network load, and how fast images appear on a viewer.

DICOM, the medical imaging standard, already supports several lossless formats — JPEG 2000, HTJ2K, JPEG-LS, and a basic run-length scheme. Each makes a different trade-off between how small the file gets, how fast it decodes, and how complex it is to implement. JPEG 2000 and HTJ2K compress well but rely on fairly heavy machinery. JPEG-LS usually gets the smallest files. Plain run-length coding is simple but barely compresses.

1.1 The core problem: everything assumes 8-bit data

Here is the observation the whole paper is built on. The popular entropy coders — Huffman, arithmetic coding, the LZ77 family, and the FSE/ANS coder inside Zstandard — were all designed for **8-bit byte streams**, where there are only 256 possible symbols. Medical images break that assumption: a 16-bit pixel has up to 65,536 possible values.

When an 8-bit coder meets 16-bit data, it has two bad options:

1. **Split each pixel into a high byte and a low byte** and code them separately. This doubles the number of coding steps and, worse, throws away the relationship between the two bytes.
2. **Cap the alphabet at 256 values** and store anything rarer as raw data, which wastes space.

Why does splitting hurt? Consider a tiny prediction error like -2 , -1 , 0 , or $+1$. The low byte carries the real information, but the high byte is almost always the same (just a sign-extension). A byte-by-byte coder still spends bits encoding that high byte even though it's nearly fully determined by the low one. On the test images, the information shared between the two bytes ranges from 0.3 to 1.1 bits per pixel — that is the ceiling on how much a byte-splitting coder is wasting, and MIC's measured 5–22% advantage sits comfortably under that ceiling.

The residual stream. Throughout, a *residual* is the difference between a pixel and a prediction of it. Example: if the predictor guesses 1020 and the true pixel is 1023, the residual is $+3$. Smooth regions of an image produce lots of residuals near zero, and the same small values show up again and again — exactly the pattern run-length and entropy coding love.

1.2 What MIC is

MIC keeps the residuals at their native 16-bit width and combines:

1. a **simple spatial predictor**,
2. a **16-bit run-length encoder**, and
3. an **entropy coder (FSE) extended to handle large alphabets**.

Plus a multi-state decoder that improves decompression speed by letting the CPU work on several symbols at once.

1.3 Contributions, briefly

- A **16-bit-native pipeline** (Delta + 16-bit RLE + extended FSE) that beats Delta + Zstandard on every one of the 21 images.

- An **entropy coder for large alphabets** — up to 65,535 symbols, versus the 4,096-symbol limit in Zstandard’s FSE. Tables shrink automatically to fit the data (as small as ~2KB for 8-bit content).
- A **multi-state decoder** that runs 2/4/8 independent decode chains to use idle CPU capacity, with no cost to compression ratio.
- An **evaluation** against HTJ2K, JPEG-LS, and Delta + Zstandard.
- A **20 KB JavaScript decoder** plus a WebAssembly build, for client-side decoding in browsers.

Scope. This work covers lossless, single-frame, grayscale images only. Color and multi-frame extensions, lossy modes, and progressive decoding are out of scope.

2 Related Work, in One Page

The DICOM lossless codecs. JPEG 2000 uses a wavelet transform plus a sophisticated block coder (EBCOT) — great ratios, slow decode. HTJ2K swaps in a faster block coder and decodes much quicker. JPEG-LS uses a small predictor (Median Edge Detector) and Golomb-Rice coding; it’s a widely deployed standard and a strong ratio baseline. The basic DICOM run-length scheme barely compresses because it works on raw bytes with no prediction step. Research codecs like CALIC and FLIF compress well on natural images but aren’t DICOM standards and lack browser decoders.

These are the codecs MIC is measured against. The goal isn’t to dethrone them, but to explore a **simpler, residual-domain design** aimed squarely at high-bit-depth images, with strong speed and easy deployment.

Entropy coding background. Asymmetric Numeral Systems (ANS) is a family of entropy coders that replaces arithmetic coding’s interval math with simple integer state updates. FSE is a fast, table-driven version of ANS made popular by Zstandard. The catch: these implementations are tuned for byte streams (Zstandard caps FSE at 4,096 symbols).

Parallel ANS. Prior work has shown ANS decoders go faster when you run several independent streams at once (Giesen on interleaved coders; later work scaling this to many streams and to GPUs). MIC doesn’t claim to invent ANS or interleaving — it applies these known ideas inside a 16-bit-native medical codec with a specific large-alphabet design.

Table 1 places MIC against the prior work across five dimensions relevant to medical image compression.

3 How MIC Works

The pipeline is three stages, each simple enough to decode in one straight-line pass with very few branches:

Raw 16-bit pixels

- > Spatial prediction (subtract a guess based on neighbors)
- > 16-bit run-length encoding (collapse repeated values)
- > FSE entropy coding (squeeze out the remaining redundancy)
- > Compressed bytes

Table 1: Feature comparison of MIC with related codecs and entropy coding methods. “Entropy coder sees 16-bit residual as one symbol” is a property of the entropy stage, not of the codec’s overall bit-depth support: JPEG-LS and HTJ2K both handle 16-bit medical images, but their entropy stages (Golomb–Rice on context bins for JPEG-LS, bit-plane coding for HTJ2K) do not treat the predicted residual as a single 16-bit symbol. Source-line counts are order-of-magnitude context only; the reference libraries also implement standard profiles outside MIC’s current scope.

Feature	JPEG-LS	HTJ2K	Zstd	rANS	MIC
Lossless medical	Yes	Yes	General	General	Yes
Entropy coder sees 16-bit residual as one symbol	No	No	No	Configurable	Yes
Multi-state ILP	No	No	No	Yes	Yes
Browser decoder	Via WASM port	Via WASM port	Partial	No	Purpose-built JS
Source lines	5k	20k	N/A	N/A	1.1k

3.1 Spatial prediction

For each pixel, MIC predicts its value as the **average of the pixel above and the pixel to the left**, then stores only the difference (the residual). Pixels on the top edge or left edge use whatever single neighbor is available; the very first pixel is stored as-is. Division rounds down, and the decoder repeats the exact same arithmetic so reconstruction is exact.

This predictor was chosen because it’s cheap, has almost no branches on the decode side, and works well on medical images.

Other predictors we tried. We compared four predictors (left-only, average, Paeth from PNG, and MED from JPEG-LS) through the full pipeline. MED gave the best ratio (about 1.6% better than average) but decoded $1.5\text{--}2\times$ slower because of its three-way branch and a diagonal dependency that blocks the fast branch-free loop. The average predictor won on consistency and speed, so it’s the default. Table 2 summarizes the comparison.

Table 2: Predictor ablation summary: geometric means and win counts across all 21 images.

Predictor	Geo. Mean	Wins	Avg→X
Left-only	$3.38\times$	3/21	-2.3%
Average (MIC)	$3.46\times$	13/21	baseline
Paeth	$3.48\times$	2/21	$+0.5\%$
MED (JPEG-LS)	$3.52\times$	16/21	$+1.6\%$

3.2 Handling big jumps: overflow coding

Most residuals are small, but occasionally a pixel jumps far from its prediction. Rather than widen the whole stream to 32 bits to handle rare big values, MIC reserves one special code as a **delimiter**:

- Small, in-range residuals are stored directly as 16-bit codes (with zero mapped to a fixed midpoint value).
- A large, out-of-range residual is stored as the delimiter followed by the raw pixel value.

On decode, the reader checks each value: if it’s the delimiter, the next value is a raw pixel; otherwise it’s a normal residual. The ranges are designed not to overlap, so there’s never any ambiguity. In normal 8–16-bit clinical data the overflow path is rare; it exists to guarantee correctness, not for the common case. Table 3 shows the mapping for a representative bit depth.

Table 3: Overflow coding example for $d = 12$ bits (`deltaThreshold`= 2047, delimiter $D = 4095$).

Residual r	Emitted code(s)	Notes
−2	2045	in-range
−1	2046	in-range
0	2047	in-range (= threshold)
+1	2048	in-range
+2	2049	in-range
± 2047	4095, raw pixel	overflow

3.3 16-bit run-length encoding (RLE)

The residual stream is turned into two kinds of runs:

- **Same runs:** a count plus one repeated value (used when a value appears at least three times in a row).
- **Literal runs:** a count followed by that many values copied verbatim (the values need not be distinct).

A single header number tells the two apart using a midpoint trick: small header values mean “same run of this length,” large ones mean “literal run.” The longest single run is $\sim 32,767$ symbols; longer stretches are split across several headers automatically.

Why 16-bit RLE matters. Suppose 1,000 identical 16-bit residuals appear in a row. MIC encodes that as just two numbers: a count and the value. A *byte*-oriented RLE sees the high and low bytes interleaved, so no single byte repeats 1,000 times — it’s forced into two interleaved runs or falls back to literals. Working at 16 bits captures the repetition that byte-splitting hides.

Fast decode. Same runs (often 80% of all runs on smooth images) are fast-pathed: the decoder just hands back the cached value and decrements a counter, without touching the compressed data at all.

3.4 The entropy coder (FSE) for large alphabets

FSE in one paragraph. FSE keeps a single integer “state.” To decode one symbol, it looks up the current state in a table; the entry tells it which symbol to output, how many bits to read next, and how to compute the next state. So each symbol costs **one table lookup, one bit read, and one addition** — no multiplies or divides. (Encoding runs the symbols in reverse and builds the bitstream backwards; decoding reads forwards. That reversal is just how ANS works.)

MIC extends FSE to handle up to **65,535 symbols** instead of Zstandard’s 4,096, because high-bit-depth residuals genuinely need a big alphabet. It also **sizes its tables to the data**: an 8-bit image stored in 16-bit containers (common for MR) shrinks the working tables from ~ 512 KB down to about 2 KB, which keeps everything in fast cache.

Picking the table size automatically. FSE has a “tableLog” knob that sets how big and precise the probability table is. MIC picks it based on how many distinct values it sees and how much data backs each value: it uses a larger table only when there are many active symbols *and* enough data to estimate their probabilities reliably (for example, mammography images after prediction). On the test set this adaptive choice correctly turned on the larger table for exactly the nine images that benefit (1–10% better ratio), and decode speed is unaffected by the choice. Table 4 shows the ablation across selected images.

Table 4: TableLog ablation: forced TL=11/12/13 vs. adaptive selection (selected images, ARM64 reference platform).

Image	TL=11	TL=12	TL=13	Adaptive
CR	3.47×	3.63×	3.69×	3.69×
MG1	8.00×	8.57×	8.79×	8.79×
MG2	7.98×	8.55×	8.77×	8.77×
MR2	3.14×	3.24×	3.28×	3.28×
RG2	3.85×	4.12×	4.23×	4.23×
RG3	5.64×	5.95×	6.08×	6.08×

4 Going Faster: Multi-State Decoding

4.1 The bottleneck

A normal single-state ANS decoder has a problem: each symbol depends on the one before it. To decode symbol N you need the state produced by symbol $N - 1$, which means the work can’t start early. With a table lookup taking roughly 4–5 CPU cycles, the loop is stuck at about one symbol every few cycles — even though modern CPUs have 4–6 execution ports sitting mostly idle. A single decode chain uses just one of them, leaving ~75–83% of the CPU’s capacity unused.

4.2 The fix

MIC runs **several independent decoders at once** on interleaved positions. With 8 chains, chain 0 handles positions 0, 8, 16, ...; chain 1 handles 1, 9, 17, ...; and so on. The chains share nothing during decoding — each has its own state and its own stream of lookups — so the CPU’s out-of-order engine can keep all of them in flight simultaneously.

On the encoder side, the symbols are split into N interleaved groups, each compressed independently, and the resulting streams are concatenated. They all share one decode table, and each chain’s starting state is recorded in the header. The decoder just writes each chain’s output back to its positions.

4.3 How much it helps

In theory, N chains could be up to $N\times$ faster. In practice the speedup is smaller because the chains share some bookkeeping (refilling the bit reader), the unrolled loop puts pressure on the instruction cache, and reading the compressed data has its own bandwidth limit. Measured on the FSE stage alone:

- **4 states:** +46% on ARM64, +8% on AMD64 (vs. single state)

- **8 states:** a further +7% on ARM64, +2% on AMD64

with **no change to the compressed file**. ARM64 keeps improving as chains double; AMD64 flattens out earlier because its core already extracts a lot of parallelism from a single chain on its own. Table 5 shows the simple latency model against the measured ARM64 speedup.

Table 5: Multi-state decoder latency model and empirical speedup.

States (k)	Amortized latency	Theoretical	Empirical (ARM64)
1	L cy/sym	$1.0\times$	$1.0\times$
2	$L/2$ cy/sym	$2.0\times$	$1.3\times$
4	$L/4$ cy/sym	$4.0\times$	$1.5\times$
8	$L/8$ cy/sym	$8.0\times$	$1.6\times$

The decoder signals its mode with a short magic header (a reserved byte value that can’t be a valid single-state setting), so single-, two-, four-, and eight-state files all coexist without confusion. On AMD64 and ARM64 the inner loop uses each architecture’s native variable-shift instructions; other platforms get a pure-Go fallback. Table 6 summarises the on-disk layout of a single-frame stream.

Table 6: Single-frame MIC bitstream layout. All multi-byte fields are little-endian. *States k* is the number of interleaved FSE chains ($k \in \{1, 2, 4, 8\}$).

Field	Size	Meaning
Magic / mode byte	1 B	Single-state: encodes <code>tableLog</code> (value 5–17). Multi-state: <code>0xFF</code> .
Multi-state tag	1 B	For multi-state streams, one of <code>0x02</code> , <code>0x04</code> , <code>0x84</code> indicating $k = 2, 4$, or 8 .
Symbol count	4 B	Number of decoded RLE symbols.
TableLog	1 B	Decode-table size: 2^{tableLog} entries (multi-state path only; the single-state path overloads byte 0).
Normalized counts	variable	FSE normalised symbol frequencies, serialised with the standard variable-length encoding.
Per-chain initial states	$k \times 2$ B	Starting state for each FSE chain.
Per-chain stream lengths	$k \times 4$ B	Compressed-stream length per chain (omitted when $k = 1$).
Per-chain compressed streams	variable	Concatenated chain payloads.

5 How We Measured It

Dataset. 21 de-identified DICOM images across nine modalities (CT, MR, CR, X-ray, mammography, tomosynthesis, and others), from public NEMA sample sets and a public breast-tomosynthesis case. It’s a varied set but small relative to a real clinical archive — hence “on this 21-image dataset” rather than sweeping claims. Table 7 lists the images.

Baselines. HTJ2K (via OpenJPH), JPEG-LS (via CharLS), and Delta + Zstandard at its maximum level. All were called in-process (no file or subprocess overhead) for a fair speed comparison.

Table 7: Test dataset: 21 clinical DICOM images spanning nine modalities.

Image	Modality	Dimensions	Raw (MB)
MR	Brain MRI	256×256	0.13
CT	CT scan	512×512	0.50
CR	Comp. radiography	2140×1760	7.18
XR	X-ray	2048×2577	10.1
MG1	Mammography	2457×1996	9.35
MG2	Mammography	2457×1996	9.35
MG3	Mammography	4774×3064	27.3
MG4	Mammography	4096×3328	26.0
CT1	CT scan	512×512	0.50
CT2	CT scan	512×512	0.50
MG-N	Mammography	3064×4664	27.3
MR1	Brain MRI	512×512	0.50
MR2	Brain MRI	1024×1024	2.00
MR3	Brain MRI	512×512	0.50
MR4	Brain MRI	512×512	0.50
NM1	Nuclear med.	256×1024	0.50
RG1	Radiography	1841×1955	6.86
RG2	Radiography	1760×2140	7.18
RG3	Radiography	1760×1760	5.91
SC1	Sec. capture	2048×2487	9.71
XA1	Fluoroscopy	1024×1024	2.00

Metrics. Compression ratio (uncompressed $\div$ compressed), and decode and encode throughput in MB/s of uncompressed pixels.

Platforms. Two reference machines: an ARM64 (Apple M4 Pro) and an AMD64 (AWS Intel Xeon 6). Go code used the standard compiler; C components were built with `-O3`. Each benchmark ran the full image 10 times; run-to-run variance stayed under 2% on ARM64 and 5% on AMD64.

The “MIC” column in the result tables is the fastest available C decoder on each platform — the eight-state decoder, with AVX-512 acceleration of the run-length and prediction-reversal stages on AMD64. Both read identical compressed bytes; only the vector width differs.

Browser decoder. A 20 KB pure-JavaScript ES module (zero npm dependencies) plus a 2.5 MB Go WebAssembly build. The JS decoder auto-detects 1/2/4/8-state streams and was benchmarked under Node.js on the ARM64 machine.

6 Results

6.1 Compression ratio

JPEG-LS gets the smallest files on every image — its context-adaptive predictor is hard to beat on pure ratio. But MIC isn’t optimizing ratio alone; it’s after the best balance of ratio, speed, and simplicity. Across the dataset MIC averages a **3.46** ratio: about 91% of JPEG-LS (3.82) and essentially tied with HTJ2K (3.45), while decoding faster than both. Mammography compresses best of all (up to 8.79×) because its smooth tissue produces long near-zero runs. Table 8 gives the per-image ratios for all variants.

Table 8: Lossless compression ratios: all codec variants, 21 images. Bold = best ratio per row.

Image	Raw (MB)	Δ +Zstd-19	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13	1.95×	2.35×	2.38×	2.28×	2.21×	2.38×	2.52×
CT	0.50	2.05×	2.24×	1.67×	2.15×	1.96×	1.77×	2.68×
CR	7.18	3.24×	3.69×	3.81×	3.70×	3.71×	3.77×	3.96×
XR	10.1	1.43×	1.74×	1.76×	1.75×	1.76×	1.67×	1.76×
MG1	9.35	7.20×	8.79×	8.67×	8.84×	8.87×	8.25×	8.91×
MG2	9.35	7.21×	8.77×	8.65×	8.83×	8.85×	8.24×	8.90×
MG3	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.22×	2.38×
MG4	26.0	3.10×	3.47×	3.59×	3.59×	3.62×	3.51×	3.71×
CT1	0.50	2.47×	2.79×	2.49×	2.54×	2.29×	2.70×	3.19×
CT2	0.50	3.24×	3.49×	2.87×	3.11×	2.72×	3.29×	4.54×
MG-N	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.23×	2.38×
MR1	0.50	1.79×	2.09×	2.14×	2.10×	2.08×	2.13×	2.30×
MR2	2.00	2.77×	3.28×	3.34×	3.31×	3.31×	3.35×	3.52×
MR3	0.50	3.47×	3.93×	4.09×	3.89×	3.84×	4.33×	4.51×
MR4	0.50	3.58×	4.12×	4.18×	4.09×	4.03×	4.21×	4.49×
NM1	0.50	4.88×	5.15×	5.02×	5.26×	5.28×	5.76×	6.28×
RG1	6.86	1.45×	1.70×	1.70×	1.70×	1.69×	1.63×	1.72×
RG2	7.18	3.68×	4.23×	4.32×	4.28×	4.30×	4.32×	4.51×
RG3	5.91	5.46×	6.08×	6.82×	6.11×	6.12×	6.99×	7.31×
SC1	9.71	3.46×	3.71×	3.70×	3.73×	3.74×	3.85×	4.73×
XA1	2.00	4.37×	5.01×	4.94×	5.04×	5.03×	4.88×	5.39×
Geomean	—	3.03×	3.46×	3.41×	3.44×	3.38×	3.45×	3.82×

6.2 Encoding speed

MIC encodes faster than the others on most images, on both platforms, because its encoder is a single pass (predict, run-length, table-driven entropy code) while HTJ2K and JPEG-LS do more work per pixel.

- **ARM64:** MIC wins on all 21 images — roughly $1.9\text{--}3.2\times$ HTJ2K, $3\text{--}7\times$ JPEG-LS, and $35\text{--}240\times$ Delta + Zstandard (whose max-level LZ77 search is what makes it so slow).
- **AMD64:** MIC wins on 18 of 21; HTJ2K narrowly takes the two biggest mammography slabs and one radiography image. The gap is smaller here because the AMD64 baselines are themselves heavily vectorized.

Table 9 gives the per-image encoding throughput.

Table 9: Single-threaded encoding throughput (MB/s). MIC is the eight-state C encoder. Bold marks the fastest codec per row on each platform.

Image	ARM64				AMD64			
	MIC	Δ +Zstd	HTJ2K	JPEG-LS	MIC	Δ +Zstd	HTJ2K	JPEG-LS
MR	504	8	273	97	338	5	229	62
CT	529	8	200	139	292	5	206	97
CR	569	3	204	115	322	2	284	75
XR	690	3	248	125	405	3	250	84
MG1	1,168	14	456	315	546	8	647	209
MG2	1,172	14	454	315	541	8	646	209
MG3	706	3	234	143	332	2	251	104
MG4	993	7	373	215	431	6	415	150
CT1	576	11	238	178	329	7	277	125
CT2	561	8	207	172	286	5	266	125
MG-N	718	2	235	143	339	2	243	104
MR1	677	11	215	120	344	6	244	86
MR2	737	9	231	131	397	4	300	84
MR3	901	22	299	187	428	12	353	123
MR4	711	8	228	175	358	6	318	141
NM1	663	7	247	170	336	5	312	116
RG1	659	5	246	95	392	4	250	65
RG2	811	6	271	151	416	3	356	99
RG3	762	6	285	189	397	4	407	126
SC1	765	6	256	223	407	4	309	168
XA1	693	7	232	152	358	4	310	97
Geomean	740	7	273	161	376	4	311	110

6.3 Decoding speed

This is MIC’s strongest result. Its decoder is dominated by that tight FSE loop — one lookup, one bit read, one add — with no hard-to-predict branches. HTJ2K and JPEG-LS both have

data-dependent branches that cause CPU mispredictions. Zstandard is the closest relative (same predictor, also uses FSE) but works on bytes, so it pays the $2\times$ byte-splitting cost on deep data.

- **ARM64:** MIC is fastest on **20 of 21** images (Zstandard edges it out only on CT, by 1.3%). Roughly $1.5\text{--}2.6\times$ HTJ2K, $1.1\text{--}1.7\times$ Zstandard, and $2.0\text{--}4.5\times$ JPEG-LS.
- **AMD64:** MIC is fastest on **all 21** images, from a hair ahead on mammography up to $1.62\times$ on X-ray. Switching from four to eight states (plus the AVX-512 widening of the post-entropy stages) was what flipped the few remaining close calls into MIC wins.

So MIC delivers both **better ratio than Zstandard on every image** and **faster decode on nearly every image** — the signature of the 16-bit-native design. JPEG-LS keeps about a 10% ratio edge, so the real choice between them is decode speed versus storage cost. Per-image numbers are in Table 10, and the entropy stage in isolation is in Table 11.

Table 10: Single-threaded decompression throughput (MB/s). MIC is the eight-state C decoder (AVX-512 acceleration of the RLE and delta inverse stages on AMD64). Bold marks the fastest codec per row on each platform.

Image	ARM64				AMD64			
	MIC	Δ +Zstd	HTJ2K	JPEG-LS	MIC	Δ +Zstd	HTJ2K	JPEG-LS
MR	564	331	301	146	552	357	439	91
CT	466	472	356	187	453	384	329	111
CR	692	564	367	206	566	475	439	123
XR	655	572	366	150	617	508	386	88
MG1	829	691	681	547	913	599	907	332
MG2	873	684	686	557	920	599	896	333
MG3	678	473	362	201	556	358	390	117
MG4	815	581	533	259	676	466	611	157
CT1	571	502	373	238	525	425	390	143
CT2	648	575	372	222	512	456	383	133
MG-N	657	469	360	201	545	355	391	117
MR1	681	437	354	154	603	384	397	94
MR2	747	505	390	224	644	464	491	132
MR3	859	507	453	329	773	485	543	192
MR4	775	523	304	230	632	480	446	153
NM1	866	595	404	277	598	486	467	172
RG1	561	413	354	126	550	347	363	80
RG2	755	574	408	255	730	497	535	154
RG3	740	655	469	320	723	552	628	196
SC1	696	587	384	280	690	484	456	183
XA1	759	611	388	278	688	534	477	165
Geomean	733	557	386	227	602	501	475	144

Table 11: FSE decompression throughput (MB/s) for representative modalities, single-thread pure-Go decoder. Increasing the number of independent state chains exposes more instruction-level parallelism to the out-of-order engine.

Image	ARM64 (MB/s)			AMD64 (MB/s)		
	1-st	4-st	8-st	1-st	4-st	8-st
MR	612	953	765	636	899	759
CT	776	1,872	1,729	974	986	953
CR	3,051	5,749	5,521	1,426	1,581	1,638
XR	3,657	6,072	5,807	1,939	1,890	2,248
MG1	3,604	4,557	5,887	2,075	1,521	1,753
MG-N	3,588	6,378	6,307	1,714	2,294	2,473
NM1	1,798	2,535	2,740	1,091	1,274	1,522
RG1	1,786	3,492	5,412	1,198	1,717	2,104
Geomean (21)	2,349	3,422	3,667	1,464	1,575	1,614

6.4 Other MIC variants (for reference)

The main tables use one MIC column, but the codebase has several variants that exist for different deployment needs rather than to compete for the fastest number:

- **Pure Go:** the whole thing in Go with no C dependency, at roughly 40–60% of the C decoder’s speed. Useful where you can’t link C (mobile, WebAssembly, locked-down runtimes).
- **1/2/4-state decoders:** the same C decoder with fewer chains; they exist as baselines for the multi-state comparison.
- **Wavelet-based MIC:** swaps the predictor for a reversible 5/3 wavelet transform feeding the same back end. Competitive on a couple of images but generally slower on this corpus, since smooth medical images don’t reward the wavelet’s extra cost. Kept as an option, not in the main tables.

The eight-state C decoder ends up being both the simplest and the fastest configuration on this dataset.

6.5 Multi-threaded decoding

MIC can also split an image into horizontal strips and decode them on separate threads. This scales nearly linearly up to four threads on both platforms and keeps scaling to eight threads on images above ~ 5 MB, hitting **4–4.6 GB/s** on the largest mammography images. Small images don’t benefit — below about half a megabyte, thread setup costs more than it saves. We report this separately because the other codecs run single-threaded in our pipeline, so a head-to-head wouldn’t be fair. Table 12 gives the per-image strip-parallel numbers.

Practical rule of thumb: decode many small images? Use single-threaded MIC per request (already fast). Decode one big image on a multi-core box? Use strip-parallel with four to eight threads.

6.6 In the browser

The JavaScript decoder runs well. Interestingly, the **four-state** variant is 7–13% *faster* than eight-state in JS, even on the same hardware where eight states win in C. The reason is the JavaScript engine (V8): it can keep four independent chains busy but not eight, so the extra chains just add

Table 12: Multi-threaded decompression throughput (MB/s) for MIC’s strip-parallel mode. The “MIC” column reproduces the single-threaded baseline from Table 10 exactly; N -strip columns decode the same compressed bytes in N parallel threads. The MR image is too small for 8-way parallelism on both platforms ([†]).

Image	ARM64				AMD64			
	MIC	2-strip	4-strip	8-strip	MIC	2-strip	4-strip	8-strip
MR	564	663	894	702 [†]	552	329	367	294 [†]
CT	466	624	1,157	1,446	453	398	633	714
CR	692	1,052	1,976	3,605	566	758	1,320	2,169
XR	655	1,087	2,024	3,794	617	793	1,515	2,579
MG1	829	1,529	2,529	4,443	913	1,317	1,951	2,890
MG2	873	1,417	2,506	4,585	920	1,356	1,971	3,026
MG3	678	1,112	1,911	3,820	556	838	1,449	2,767
MG4	815	1,326	2,345	4,348	676	1,207	1,897	3,126
CT1	571	819	1,247	1,525	525	458	703	725
CT2	648	864	1,260	1,473	512	481	734	719
MG-N	657	1,147	2,027	3,930	545	913	1,537	2,819
MR1	681	977	1,460	1,851	603	504	761	764
MR2	747	1,137	1,911	3,103	644	778	1,171	1,534
MR3	859	1,205	1,693	2,089	773	591	788	835
MR4	775	997	1,469	1,757	632	540	796	804
NM1	866	1,147	1,682	2,133	598	523	824	762
RG1	561	677	1,203	2,150	550	603	1,101	1,890
RG2	755	1,229	2,156	3,816	730	916	1,473	2,268
RG3	740	1,335	2,343	4,168	723	971	1,618	2,730
SC1	696	1,311	2,206	3,971	690	1,024	1,673	2,541
XA1	759	1,214	1,881	3,191	688	792	1,246	1,748

overhead without shortening the critical path. The eight-state path still works in JS — it lets one decoder accept C-encoded files without conversion — but four states is the sweet spot for the browser.

Concretely, a 9.35 MB mammography image decodes in 19.8 ms (473 MB/s) using eight workers with four-state strips. That means a web DICOM viewer can pull a `.mic` file straight from object storage and decode it client-side, with no server doing compression or format conversion. (These numbers are from Node.js; full in-browser benchmarking is future work.) Tables 13 and 14 give the JS numbers.

Table 13: JavaScript decoder throughput on the ARM64 reference (Apple M4 Pro, Node.js v22.16), for the four-state and eight-state FSE variants. Median over 20 iterations after three warm-up runs.

Image	Dimensions	Ratio	4-state		8-state	
			ms	MB/s	ms	MB/s
MR	256×256	2.35×	2.5	50	3.3	38
CT	512×512	2.24×	11.9	42	14.1	36
CR	2140×1760	3.69×	136.4	53	151.4	47
MG1	2457×1996	8.79×	68.1	137	78.3	120
MG3	4774×3064	2.29×	514.5	54	563.7	49

Table 14: PICS parallel JavaScript decoder (`worker_threads`) on the ARM64 reference, with four-state and eight-state FSE per strip. Worker count was swept over $\{1, 2, 4, 8, 14\}$; the best wall-time for each variant is reported.

Image	Strips	4-state strips		8-state strips	
		ms	MB/s	ms	MB/s
MR	4	1.1	111	1.3	97
CT	4	4.2	120	4.7	108
CR	8	33.4	215	31.8	226
MG1	8	19.8	473	20.5	457

7 Discussion

7.1 Reading the results together

Three things tie the numbers together:

1. **MIC’s ratio edge over Zstandard** (5–22%, 14% average) comes mainly from working on 16-bit residuals directly instead of splitting them into bytes — and that same choice removes the per-byte branches that slow decoding.
2. **On the very largest mammography images on AMD64**, HTJ2K’s block coder regains the speed lead by amortizing its setup over many pixels; MIC stays within 0–20% there.
3. **Multi-state decoding** is what makes MIC’s decoder competitive with HTJ2K’s vectorized block coder, and it costs nothing in file size.

7.2 Which codec to pick

- **Low-latency viewing (PACS, thumbnails, tomosynthesis playback):** single-threaded MIC is a sensible default ($\sim 600\text{--}900$ MB/s per request); strip-parallel MIC for one big image at a time.
- **Smallest archival files:** JPEG-LS, which keeps $\sim 10\%$ better ratio — at the cost of decoding $1.6\text{--}5.7\times$ slower on ARM64.
- **DICOM transfer-syntax compatibility required:** HTJ2K, since it’s a standard (and it’s specifically strong on big mammography on AMD64).
- **No native code allowed (browsers, sandboxes):** the pure-Go reference or the 20 KB JavaScript decoder, at about half the C decoder’s speed.

A more structured comparison (a Pareto view of ratio vs. decode speed, plus a weighted scoring matrix over ratio, speed, platform coverage, and deployment ease) puts MIC and JPEG-LS on the efficiency frontier — MIC for throughput, JPEG-LS for ratio — with HTJ2K and Zstandard dominated on those two axes. HTJ2K still wins whenever standard compatibility is a hard requirement that the scoring doesn’t capture. Table 15 reports the weighted matrix.

Table 15: Codec selection matrix. Each criterion is normalised so the best codec on that row scores 1.00. Quantitative rows are geometric means across the 21-image set. Browser compatibility: shipped purpose-built JS or WASM decoder used in production = 1.00; WASM port of the native library used in clinical viewers = 0.85; no browser path = 0.0. Deployment ease: zero external runtime dependencies = 1.00; ubiquitous system dependency = 0.90; C/C++ library requiring CMake build and link integration = 0.70. Weight columns w_P , w_A , w_B reflect typical priorities for (P) PACS viewer / low-latency decode, (A) archival storage, (B) browser delivery.

Criterion	w_P	w_A	w_B	MIC	HTJ2K	JPEG-LS
Compression ratio	0.20	0.50	0.10	0.91	0.90	1.00
Decompression throughput (ARM64)	0.30	0.10	0.15	1.00	0.53	0.31
Encoding throughput (ARM64)	0.05	0.10	0.05	1.00	0.37	0.22
AMD64 native	0.10	0.05	0.05	1.00	1.00	1.00
ARM64 native	0.10	0.05	0.05	1.00	1.00	1.00
Browser compatibility	0.05	0.00	0.40	1.00	0.85	0.85
Deployment ease	0.20	0.20	0.20	1.00	0.70	0.70
Weighted score (P) viewer	—	—	—	0.98	0.74	0.69
Weighted score (A) archival	—	—	—	0.96	0.78	0.80
Weighted score (B) browser	—	—	—	0.99	0.78	0.74

7.3 Limitations

- **Small dataset.** 21 images across nine modalities is varied but far from a clinical archive; broader validation is needed.
- **Tuning on the test set.** The adaptive table-size thresholds and the predictor choice were informed by this dataset. The rules are simple (two thresholds, one predictor), which limits overfitting risk, but larger studies should confirm they generalize.
- **Baseline coverage.** Stronger byte-oriented baselines (Delta + byte/bit-shuffle + Zstd, DICOM RLE) would more sharply isolate the value of the 16-bit entropy stage; left to future work.

- **Reporting.** Tables give single representative throughput values rather than full confidence intervals.
- **Not yet a DICOM transfer syntax.** MIC works on the extracted pixel array; real PACS integration would need a private encapsulation or a standardization step.

8 Conclusion

MIC is a 16-bit-native lossless codec for medical images: spatial prediction, 16-bit run-length coding, a large-alphabet table-based entropy coder, and a multi-state decoder for speed. On 21 DICOM images it reaches an average ratio of 3.46 (tied with HTJ2K, $\sim 91\%$ of JPEG-LS) while being the fastest decoder on 20/21 images on ARM64 and all 21 on AMD64, and the fastest encoder on all 21 ARM64 images and 18/21 on AMD64. A 20 KB JavaScript decoder and a WebAssembly build show the format can be decoded client-side in a browser.

Future work: larger multi-hospital datasets; stronger shuffle-based baselines; full statistical reporting; an ARM64 vector kernel for the wavelet variant; and benchmarking the JavaScript decoder in real browsers.

MIC is open source: <https://github.com/pappuks/medical-image-codec>